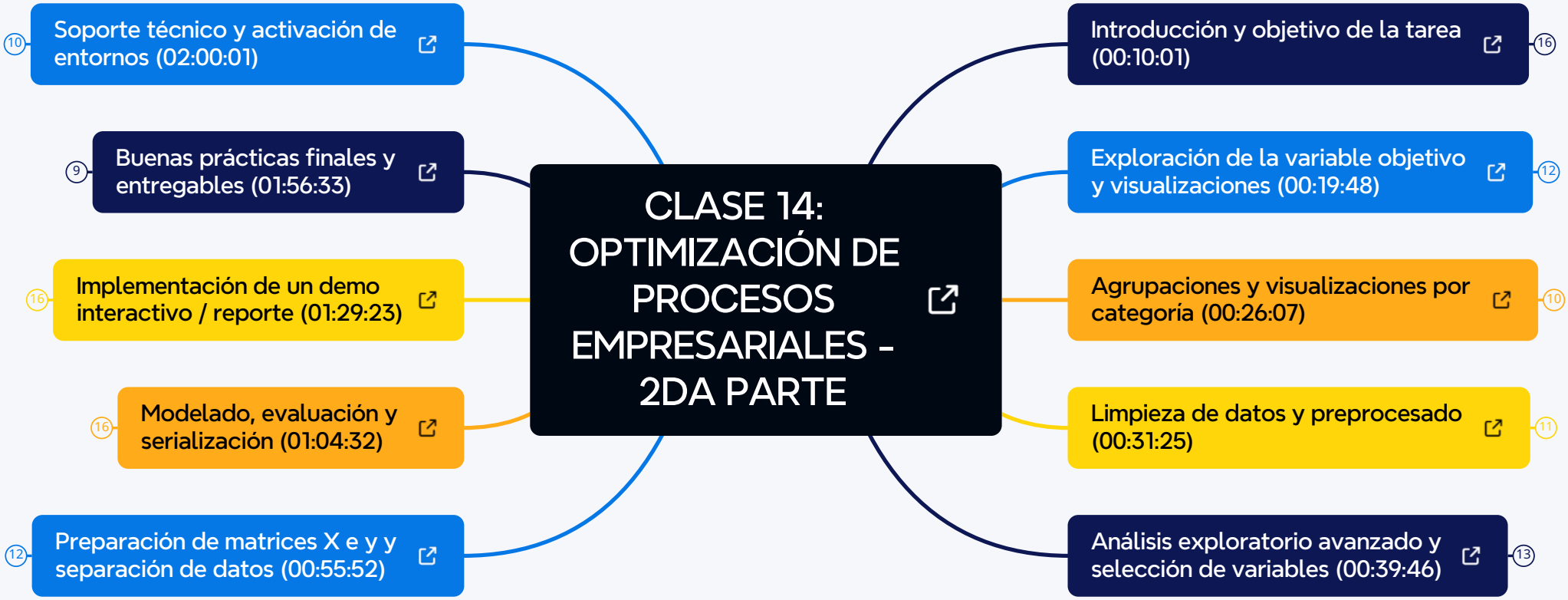
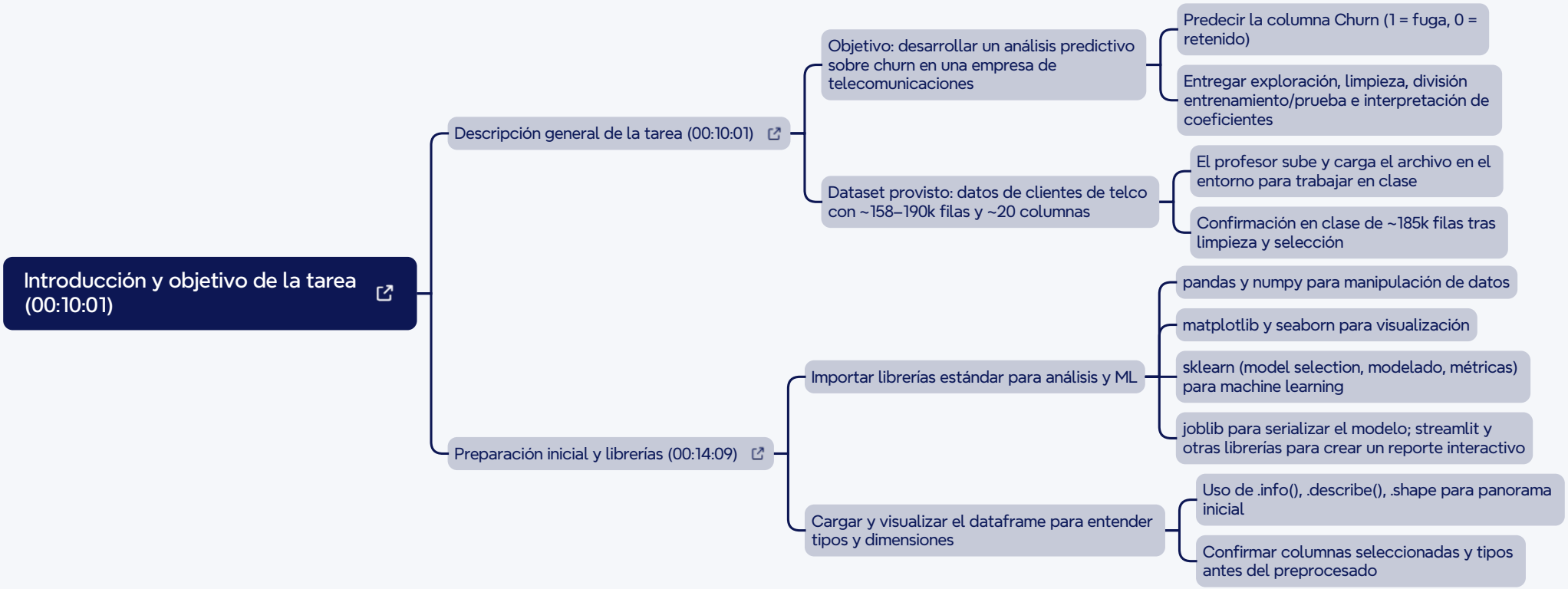
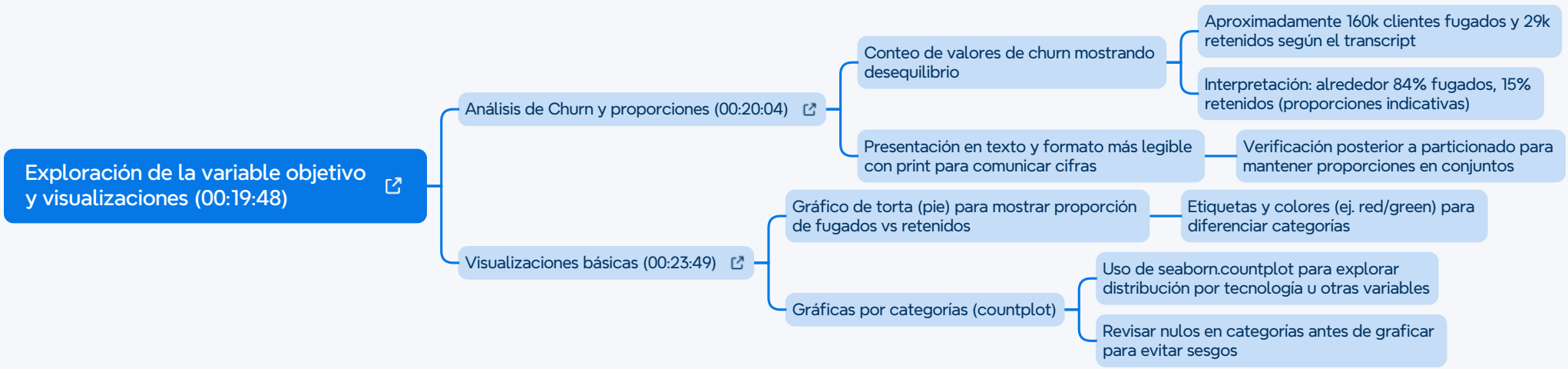
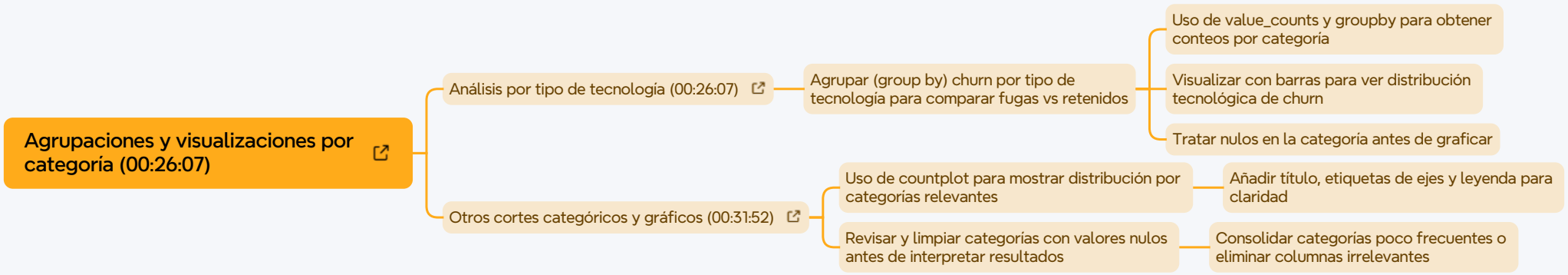
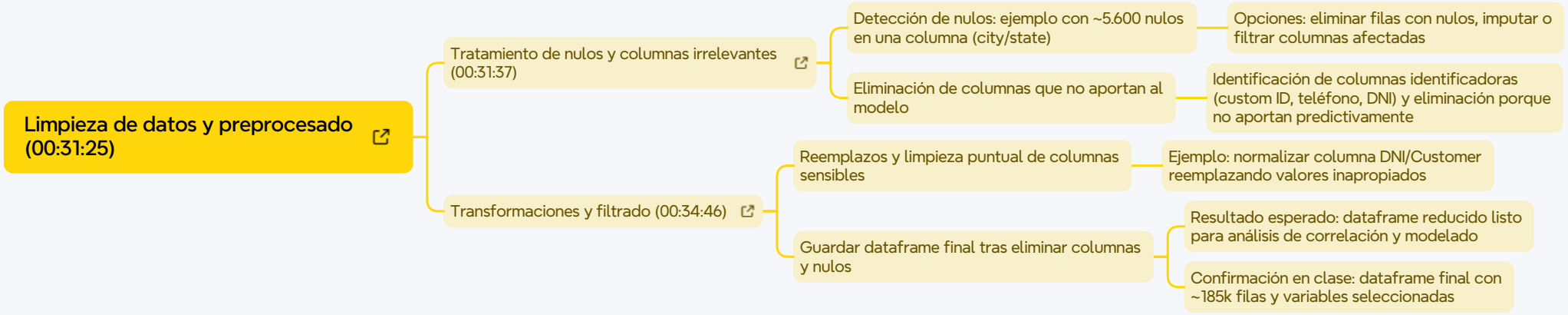
EDU MIN

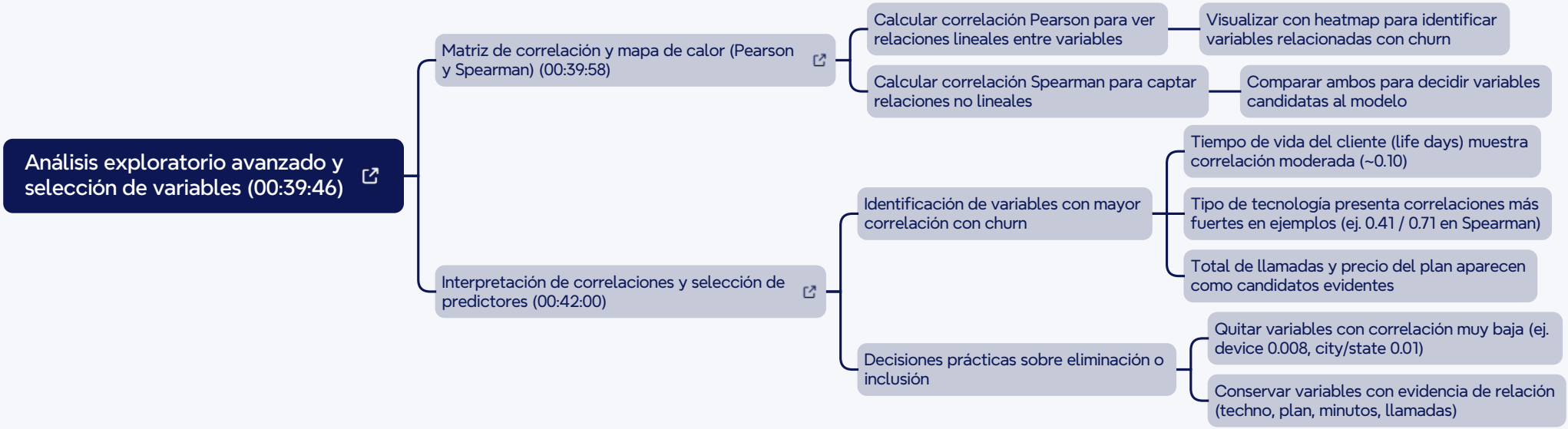

CLASE 14: OPTIMIZACIÓN DE PROCESOS EMPRESARIALES - 2DA PARTE
Introducción y objetivo de la tarea (00:10:01)
16
Exploración de la variable objetivo y visualizaciones (00:19:48)
12
Agrupaciones y visualizaciones por categoría (00:26:07)
10
Limpieza de datos y preprocesado (00:31:25)
11
Análisis exploratorio avanzado y selección de variables (00:39:46)
13
Soporte técnico y activación de entornos (02:00:01)
10
Buenas prácticas finales y entregables (01:56:33)
9
Implementación de un demo interactivo / reporte (01:29:23)
16
Modelado, evaluación y serialización (01:04:32)
16
Preparación de matrices X e y y separación de datos (00:55:52)
12

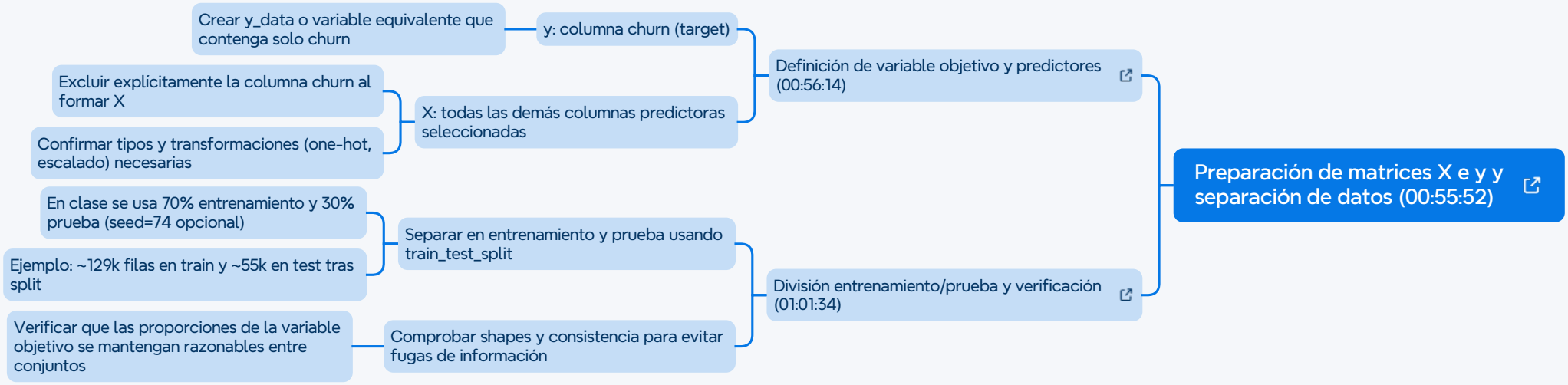

Introducción y objetivo de la tarea (00:10:01)
Descripción general de la tarea (00:10:01)
Objetivo: desarrollar un análisis predictivo sobre churn en una empresa de telecomunicaciones
Predecir la columna Churn (1 = fuga, 0 = retenido)
Entregar exploración, limpieza, división entrenamiento/prueba e interpretación de coeficientes
Dataset provisto: datos de clientes de telco con ~158–190k filas y ~20 columnas
El profesor sube y carga el archivo en el entorno para trabajar en clase
Confirmación en clase de ~185k filas tras limpieza y selección
Preparación inicial y librerías (00:14:09)
Importar librerías estándar para análisis y ML
pandas y numpy para manipulación de datos
matplotlib y seaborn para visualización
sklearn (model selection, modelado, métricas) para machine learning
joblib para serializar el modelo; streamlit y otras librerías para crear un reporte interactivo
Cargar y visualizar el dataframe para entender tipos y dimensiones
Uso de .info(), .describe(), .shape para panorama inicial
Confirmar columnas seleccionadas y tipos antes del preprocesado

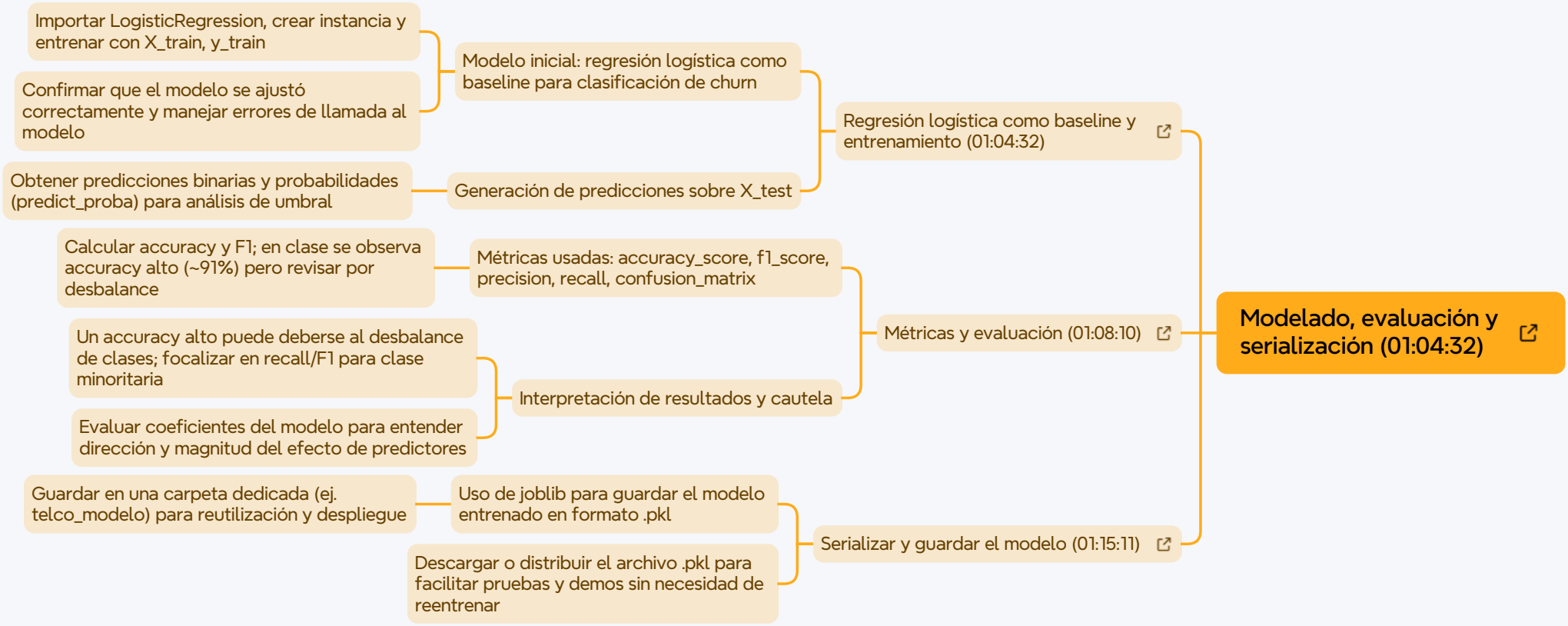

Exploración de la variable objetivo y visualizaciones (00:19:48)
Análisis de Churn y proporciones (00:20:04)
Conteo de valores de churn mostrando desequilibrio
Aproximadamente 160k clientes fugados y 29k retenidos según el transcript
Interpretación: alrededor 84% fugados, 15% retenidos (proporciones indicativas)
Presentación en texto y formato más legible con print para comunicar cifras
Verificación posterior a particionado para mantener proporciones en conjuntos
Visualizaciones básicas (00:23:49)
Gráfico de torta (pie) para mostrar proporción de fugados vs retenidos
Etiquetas y colores (ej. red/green) para diferenciar categorías
Gráficas por categorías (countplot)
Uso de seaborn.countplot para explorar distribución por tecnología u otras variables
Revisar nulos en categorías antes de graficar para evitar sesgos

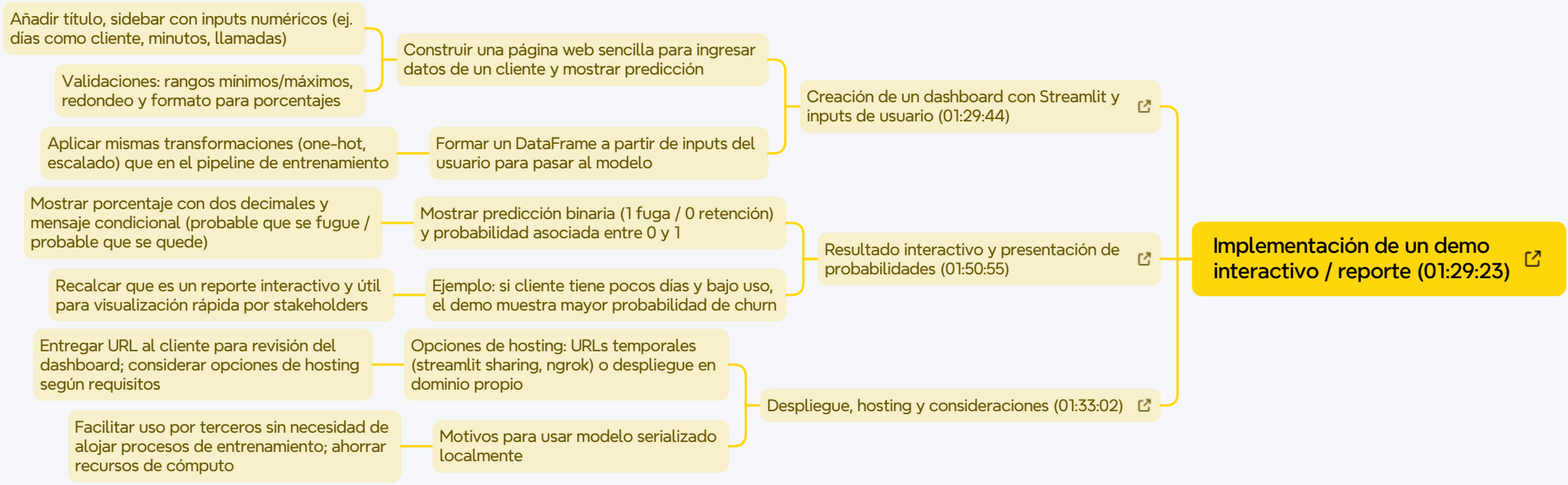

Agrupaciones y visualizaciones por categoría (00:26:07)
Análisis por tipo de tecnología (00:26:07)
Agrupar (group by) churn por tipo de tecnología para comparar fugas vs retenidos
Uso de value_counts y groupby para obtener conteos por categoría
Visualizar con barras para ver distribución tecnológica de churn
Tratar nulos en la categoría antes de graficar
Otros cortes categóricos y gráficos (00:31:52)
Uso de countplot para mostrar distribución por categorías relevantes
Añadir título, etiquetas de ejes y leyenda para claridad
Revisar y limpiar categorías con valores nulos antes de interpretar resultados
Consolidar categorías poco frecuentes o eliminar columnas irrelevantes

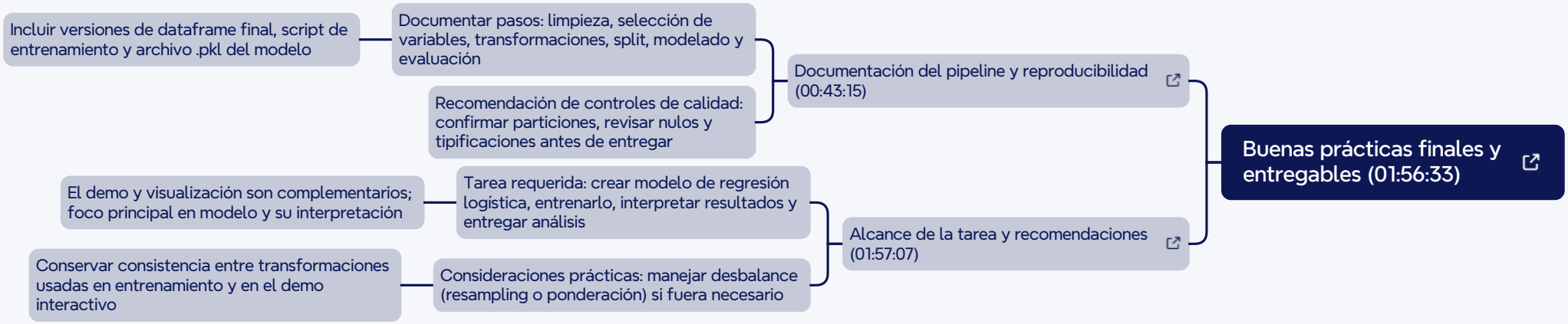

Limpieza de datos y preprocesado (00:31:25)
Tratamiento de nulos y columnas irrelevantes (00:31:37)
Detección de nulos: ejemplo con ~5.600 nulos en una columna (city/state)
Opciones: eliminar filas con nulos, imputar o filtrar columnas afectadas
Eliminación de columnas que no aportan al modelo
Identificación de columnas identificadoras (custom ID, teléfono, DNI) y eliminación porque no aportan predictivamente
Transformaciones y filtrado (00:34:46)
Reemplazos y limpieza puntual de columnas sensibles
Ejemplo: normalizar columna DNI/Customer reemplazando valores inapropiados
Guardar dataframe final tras eliminar columnas y nulos
Resultado esperado: dataframe reducido listo para análisis de correlación y modelado
Confirmación en clase: dataframe final con ~185k filas y variables seleccionadas

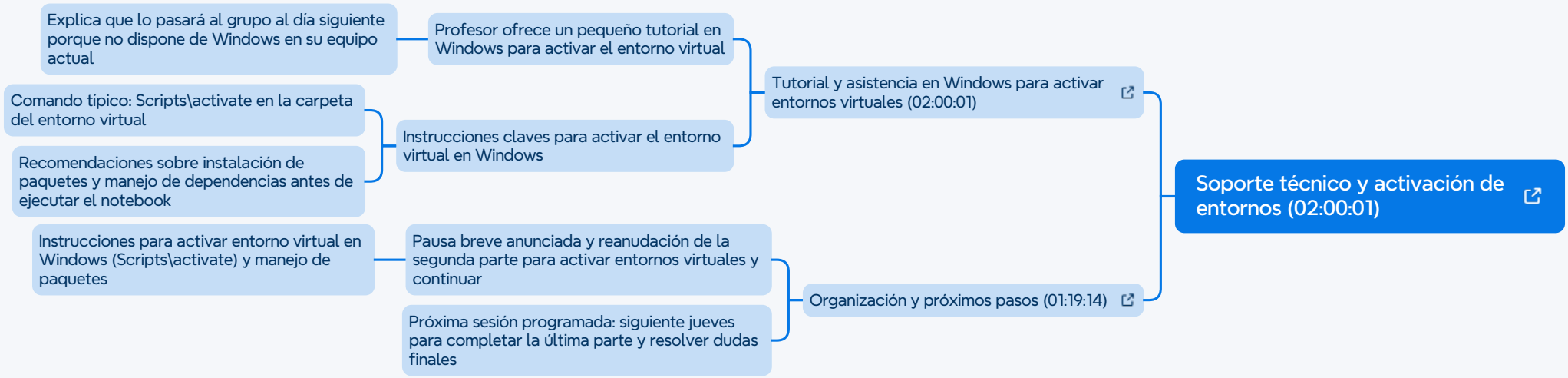

Análisis exploratorio avanzado y selección de variables (00:39:46)
Matriz de correlación y mapa de calor (Pearson y Spearman) (00:39:58)
Calcular correlación Pearson para ver relaciones lineales entre variables
Visualizar con heatmap para identificar variables relacionadas con churn
Calcular correlación Spearman para captar relaciones no lineales
Comparar ambos para decidir variables candidatas al modelo
Interpretación de correlaciones y selección de predictores (00:42:00)
Identificación de variables con mayor correlación con churn
Tiempo de vida del cliente (life days) muestra correlación moderada (~0.10)
Tipo de tecnología presenta correlaciones más fuertes en ejemplos (ej. 0.41 / 0.71 en Spearman)
Total de llamadas y precio del plan aparecen como candidatos evidentes
Decisiones prácticas sobre eliminación o inclusión
Quitar variables con correlación muy baja (ej. device 0.008, city/state 0.01)
Conservar variables con evidencia de relación (techno, plan, minutos, llamadas)


Preparación de matrices X e y y separación de datos (00:55:52)
Definición de variable objetivo y predictores (00:56:14)
División entrenamiento/prueba y verificación (01:01:34)
y: columna churn (target)
Crear y_data o variable equivalente que contenga solo churn
X: todas las demás columnas predictoras seleccionadas
Excluir explícitamente la columna churn al formar X
Confirmar tipos y transformaciones (one-hot, escalado) necesarias
Separar en entrenamiento y prueba usando train_test_split
En clase se usa 70% entrenamiento y 30% prueba (seed=74 opcional)
Ejemplo: ~129k filas en train y ~55k en test tras split
Comprobar shapes y consistencia para evitar fugas de información
Verificar que las proporciones de la variable objetivo se mantengan razonables entre conjuntos


Modelado, evaluación y serialización (01:04:32)
Regresión logística como baseline y entrenamiento (01:04:32)
Modelo inicial: regresión logística como baseline para clasificación de churn
Importar LogisticRegression, crear instancia y entrenar con X_train, y_train
Confirmar que el modelo se ajustó correctamente y manejar errores de llamada al modelo
Generación de predicciones sobre X_test
Obtener predicciones binarias y probabilidades (predict_proba) para análisis de umbral
Métricas y evaluación (01:08:10)
Métricas usadas: accuracy_score, f1_score, precision, recall, confusion_matrix
Calcular accuracy y F1; en clase se observa accuracy alto (~91%) pero revisar por desbalance
Interpretación de resultados y cautela
Un accuracy alto puede deberse al desbalance de clases; focalizar en recall/F1 para clase minoritaria
Evaluar coeficientes del modelo para entender dirección y magnitud del efecto de predictores
Serializar y guardar el modelo (01:15:11)
Uso de joblib para guardar el modelo entrenado en formato .pkl
Guardar en una carpeta dedicada (ej. telco_modelo) para reutilización y despliegue
Descargar o distribuir el archivo .pkl para facilitar pruebas y demos sin necesidad de reentrenar


Implementación de un demo interactivo / reporte (01:29:23)
Creación de un dashboard con Streamlit y inputs de usuario (01:29:44)
Construir una página web sencilla para ingresar datos de un cliente y mostrar predicción
Añadir título, sidebar con inputs numéricos (ej. días como cliente, minutos, llamadas)
Validaciones: rangos mínimos/máximos, redondeo y formato para porcentajes
Formar un DataFrame a partir de inputs del usuario para pasar al modelo
Aplicar mismas transformaciones (one-hot, escalado) que en el pipeline de entrenamiento
Resultado interactivo y presentación de probabilidades (01:50:55)
Mostrar predicción binaria (1 fuga / 0 retención) y probabilidad asociada entre 0 y 1
Mostrar porcentaje con dos decimales y mensaje condicional (probable que se fugue / probable que se quede)
Ejemplo: si cliente tiene pocos días y bajo uso, el demo muestra mayor probabilidad de churn
Recalcar que es un reporte interactivo y útil para visualización rápida por stakeholders
Despliegue, hosting y consideraciones (01:33:02)
Opciones de hosting: URLs temporales (streamlit sharing, ngrok) o despliegue en dominio propio
Entregar URL al cliente para revisión del dashboard; considerar opciones de hosting según requisitos
Motivos para usar modelo serializado localmente
Facilitar uso por terceros sin necesidad de alojar procesos de entrenamiento; ahorrar recursos de cómputo


Buenas prácticas finales y entregables (01:56:33)
Documentación del pipeline y reproducibilidad (00:43:15)
Documentar pasos: limpieza, selección de variables, transformaciones, split, modelado y evaluación
Incluir versiones de dataframe final, script de entrenamiento y archivo .pkl del modelo
Recomendación de controles de calidad: confirmar particiones, revisar nulos y tipificaciones antes de entregar
Alcance de la tarea y recomendaciones (01:57:07)
Tarea requerida: crear modelo de regresión logística, entrenarlo, interpretar resultados y entregar análisis
El demo y visualización son complementarios; foco principal en modelo y su interpretación
Consideraciones prácticas: manejar desbalance (resampling o ponderación) si fuera necesario
Conservar consistencia entre transformaciones usadas en entrenamiento y en el demo interactivo


Soporte técnico y activación de entornos (02:00:01)
Tutorial y asistencia en Windows para activar entornos virtuales (02:00:01)
Profesor ofrece un pequeño tutorial en Windows para activar el entorno virtual
Explica que lo pasará al grupo al día siguiente porque no dispone de Windows en su equipo actual
Instrucciones claves para activar el entorno virtual en Windows
Comando típico: Scripts\activate en la carpeta del entorno virtual
Recomendaciones sobre instalación de paquetes y manejo de dependencias antes de ejecutar el notebook
Organización y próximos pasos (01:19:14)
Pausa breve anunciada y reanudación de la segunda parte para activar entornos virtuales y continuar
Instrucciones para activar entorno virtual en Windows (Scripts\activate) y manejo de paquetes
Próxima sesión programada: siguiente jueves para completar la última parte y resolver dudas finales